

DATA STRUCTURES

Complete Exam Preparation Notes

Questions · Answers · Algorithms · Complexity Analysis

■ Topics Covered

Stacks & Queues | Linked Lists | Trees & Graphs Sorting & Searching | Dynamic Programming |
AVL Trees

This document is compiled from class notes and covers all topics as highlighted by your instructor. Every question has a detailed answer written for full exam marks.

STACKS & QUEUES

Overflow · Underflow · Circular Queue · Push/Pop

Q. What is the overflow and underflow condition of a Stack or Queue? What will you do when overflow or underflow is happening?

STACK – Overflow & Underflow

Overflow Condition (Stack):

Overflow occurs when you try to **push an element onto a full stack**. For an array-based stack of size N, overflow happens when **top == N-1**.

Condition: `top == MAX_SIZE - 1` → OVERFLOW

Underflow Condition (Stack):

Underflow occurs when you try to **pop from an empty stack**, i.e., when **top == -1**.

Condition: `top == -1` → UNDERFLOW

What to do when Overflow/Underflow happens:

- Overflow → Print error message: *"Stack Overflow! Cannot push."* OR dynamically resize the array (double capacity).
- Underflow → Print error message: *"Stack Underflow! Cannot pop."* OR return a sentinel value (e.g., -1 or NULL).

QUEUE – Overflow & Underflow

For a linear queue of size N:

Overflow: `rear == MAX_SIZE - 1`

Underflow: `front == -1` OR `front > rear`

- On overflow → reject enqueue / resize.
- On underflow → reject dequeue / return error.

■ In a **CIRCULAR QUEUE**: Overflow when **`(rear+1) % MAX_SIZE == front`**

Q. Given: Stack or Queue with code – Explain push, pop, etc.

PUSH Operation (Stack)

```
void push(int stack[], int *top, int item) {
    if (*top == MAX-1) { printf('Stack Overflow'); return; }
    stack[++(*top)] = item;
}
```

POP Operation (Stack)

```
int pop(int stack[], int *top) {
```

```

if (*top == -1) { printf('Stack Underflow'); return -1; }
return stack[(*top)--];
}

```

Trace Example: Push(stack, AB) then Pop(stack, item):

After Push → top increments and element is stored at new top.

After Pop → element at top is returned, top decrements.

Final value of variable: the popped item (last pushed element) due to LIFO (Last In First Out) nature.

Q. Given a stack with elements 1–8: arrange even numbers on TOP and odd numbers on BOTTOM of the stack.

Stack initially: [1, 2, 3, 4, 5, 6, 7, 8] (1 at bottom, 8 at top)

Approach (using an auxiliary stack):

- Step 1: Pop all elements from original stack into a temporary list.
- Step 2: Separate odd and even numbers.
- Step 3: Push ODD numbers first (they go to bottom).
- Step 4: Push EVEN numbers on top of odds.

Result:

Position	Value	Type
TOP → 8	8	Even
7	6	Even
6	4	Even
5	2	Even
4	7	Odd
3	5	Odd
2	3	Odd
Bottom → 1	1	Odd

■ Even: 2,4,6,8 → Top | Odd: 1,3,5,7 → Bottom

Q. Given a queue with elements 1–8: arrange such that odd and even numbers are side by side (together).

Queue initially: | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | (Front→Left)

Approach:

- Dequeue all elements; separate into two queues: Q_odd and Q_even.
- Interleave: dequeue one from Q_odd, one from Q_even, alternately.

Result: | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | → | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |

Rearranged: | 2 | 1 | 4 | 3 | 6 | 5 | 8 | 7 | (even,odd pairs)

■ Alternatively odds first block then evens block: |1|3|5|7|2|4|6|8|

Q. What is a Circular Queue? Implement it using an array with example.

A **Circular Queue** overcomes the limitation of a linear queue where space is wasted after deletions. The rear pointer wraps around to the beginning when it reaches the end of the array.

Key Formulas

Enqueue: $\text{rear} = (\text{rear} + 1) \% \text{MAX_SIZE}$

Dequeue: $\text{front} = (\text{front} + 1) \% \text{MAX_SIZE}$

Full: $(\text{rear} + 1) \% \text{MAX_SIZE} == \text{front}$

Empty: $\text{front} == -1$

Time Complexity

Operation	Time Complexity
Enqueue	O(1)
Dequeue	O(1)
Peek/Front	O(1)
Is Empty/Full	O(1)

CHAPTER 2

LINKED LIST

Add · Delete · Time Complexity for all cases

Q. What is a Linked List? Explain operations: add to first, last, and specific location with time complexity of each.

A **Linked List** is a linear data structure where elements (nodes) are stored in non-contiguous memory. Each node contains **data** and a **pointer to the next node**.

Time Complexity of LinkedList Operations

Operation	Best Case	Average Case	Worst Case
Add at Beginning	$O(1)$	$O(1)$	$O(1)$
Add at End (no tail ptr)	$O(n)$	$O(n)$	$O(n)$
Add at End (with tail ptr)	$O(1)$	$O(1)$	$O(1)$
Add at Specific Location	$O(1)$	$O(n)$	$O(n)$
Delete from Beginning	$O(1)$	$O(1)$	$O(1)$
Delete from End	$O(n)$	$O(n)$	$O(n)$
Delete at Specific Location	$O(1)$	$O(n)$	$O(n)$
Search	$O(1)$	$O(n)$	$O(n)$

Memory Allocation Comparison

Data Structure	Memory Type	Size	Access
Array	Contiguous (Static)	Fixed	$O(1)$ Random
Linked List	Non-contiguous (Dynamic)	Flexible	$O(n)$ Sequential
Stack (array)	Contiguous	Fixed/Dynamic	$O(1)$ Top only
Queue (array)	Contiguous	Fixed/Dynamic	$O(1)$ Front/Rear
Heap	Contiguous (array-based)	Fixed	$O(1)$ Root
Tree	Non-contiguous	Dynamic	$O(\log n)$ avg

SEARCHING ALGORITHMS

Linear · Binary · Jump Search + Complexity

Q. Explain different searching algorithms: Linear Search, Binary Search, Jump Search with their time and space complexities.

1. Linear Search

Searches each element one by one from start to end. Works on both **sorted and unsorted** arrays.

Time: $O(n)$ | Space: $O(1)$

- Best Case: $O(1)$ – element found at first position.
- Worst Case: $O(n)$ – element at last position or not found.

2. Binary Search

Works only on **sorted arrays**. Repeatedly divides the search interval in half.

$mid = (low + high) / 2$

Time: $O(\log n)$ | Space: $O(1)$ iterative, $O(\log n)$ recursive

- Best Case: $O(1)$ – element is at mid.
- Worst Case: $O(\log n)$

3. Jump Search

Works on **sorted arrays**. Jumps ahead by fixed steps $\sqrt{n}$, then does linear search in the block.

Optimal block size = $\sqrt{n}$

Time: $O(\sqrt{n})$ | Space: $O(1)$

Comparison Table

Algorithm	Requirement	Best	Average	Worst	Space
Linear Search	None	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Binary Search	Sorted	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$
Jump Search	Sorted	$O(1)$	$O(\sqrt{n})$	$O(\sqrt{n})$	$O(1)$

SORTING ALGORITHMS

Bubble · Selection · Insertion · Merge · Quick · Heap

Q. Explain sorting algorithms with time complexity, space complexity, and pseudo code. Sort/arrange complexities in ascending or descending order.

1. Bubble Sort

Repeatedly swaps adjacent elements if they are in wrong order. In each pass the largest element bubbles to the end.

Best: $O(n)$ | Average: $O(n^2)$ | Worst: $O(n^2)$ | Space: $O(1)$

Pseudo Code:

```
for i = 0 to n-1:
  for j = 0 to n-i-2:
    if arr[j] > arr[j+1]: swap(arr[j], arr[j+1])
```

2. Selection Sort

Finds the minimum element and places it at the beginning. Repeat for remaining elements.

Best/Average/Worst: $O(n^2)$ | Space: $O(1)$

```
for i = 0 to n-1:
  min_idx = i
  for j = i+1 to n: if arr[j] < arr[min_idx]: min_idx = j
  swap(arr[i], arr[min_idx])
```

3. Insertion Sort

Builds sorted array one element at a time by inserting each new element into its correct position.

Best: $O(n)$ | Average: $O(n^2)$ | Worst: $O(n^2)$ | Space: $O(1)$

Step-by-step: Given sorted data → insert one new element → shift elements right until correct position found → insert.

```
for i = 1 to n-1:
  key = arr[i]; j = i-1
  while j >= 0 and arr[j] > key: arr[j+1] = arr[j]; j--
  arr[j+1] = key
```

4. Merge Sort

Divide and Conquer algorithm. Divides array into halves, recursively sorts each half, then merges the sorted halves.

Best/Average/Worst: $O(n \log n)$ | Space: $O(n)$

Algorithm (two parts):

- **Merging:** Merge two sorted subarrays into one sorted array.
- **Merge Sort:** Recursively split until single elements, then merge.

```
MergeSort(arr, l, r):
```

```
if l < r:
```

```
mid = (l + r) / 2
```

```
MergeSort(arr, l, mid)
```

```
MergeSort(arr, mid+1, r)
```

```
Merge(arr, l, mid, r)
```

5. Quick Sort

Divide and Conquer. Picks a **pivot**, partitions array so elements smaller than pivot go left and larger go right. Recursively sorts both parts.

Best/Average: $O(n \log n)$ | Worst: $O(n^2)$ | Space: $O(\log n)$

Algorithm (two parts):

- **Partition:** Rearrange elements around pivot.
- **Quick Sort:** Recursively apply to left and right partitions.

```
QuickSort(arr, low, high):
```

```
if low < high:
```

```
pi = Partition(arr, low, high)
```

```
QuickSort(arr, low, pi-1)
```

```
QuickSort(arr, pi+1, high)
```

Pivot selection strategy:

- First element / Last element / Middle element / Random element / Median-of-three

■ **Worst case $O(n^2)$ occurs when pivot is always smallest or largest element (already sorted array with first/last pivot).**

6. Heap Sort

Uses a **max-heap** data structure. First builds heap, then repeatedly extracts maximum.

Best/Average/Worst: $O(n \log n)$ | Space: $O(1)$

MASTER COMPARISON TABLE

Algorithm	Best	Average	Worst	Space	Stable
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No

Complexities in ASCENDING order (fastest → slowest):

$O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$

■ ■ For exam: Merge Sort and Heap Sort are always $O(n \log n)$ worst case. Quick Sort worst is $O(n^2)$.

Q. Find the time complexity of: single loop, multi loop (nested), and 3× nested loop.

Loop Type	Example	Time Complexity
Single loop	for i in range(n):	$O(n)$
Double (nested) loop	for i ... for j ...:	$O(n^2)$
Triple (3×) loop	for i ... for j ... for k ...:	$O(n^3)$
Logarithmic loop	while n > 1: n = n/2	$O(\log n)$
n × log n loop	outer $O(n)$, inner $O(\log n)$	$O(n \log n)$

Q. What is time complexity of recursions?

Use the **Master Theorem** or recurrence relation:

$$T(n) = aT(n/b) + f(n)$$

- Merge Sort: $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$
 - Binary Search: $T(n) = T(n/2) + O(1) \rightarrow O(\log n)$
 - Fibonacci (naive): $T(n) = T(n-1) + T(n-2) \rightarrow O(2^n)$
-

CHAPTER 5

TREES

Binary Tree · BST · Tree Traversals · AVL Trees

Q. What is a Binary Tree? Explain Pre-order, In-order, and Post-order traversals with traversal sequences.

Tree Traversals

Traversal	Order	Sequence Formula	Use Case
Pre-order	Root → Left → Right	Root L R	Copy tree, Prefix expression
In-order	Left → Root → Right	L Root R	Sorted output from BST
Post-order	Left → Right → Root	L R Root	Delete tree, Postfix expression

Which traversal pairs can reconstruct a unique tree?

Combination	Unique Tree?	Reason
Pre-order + In-order	■ YES	In-order partitions left/right
Post-order + In-order	■ YES	In-order partitions left/right
Pre-order + Post-order	■ NO	Cannot determine left/right subtrees uniquely

Q. What is Binary Search Tree (BST)? Difference between Binary Search and Binary Tree.

Feature	Binary Tree	Binary Search Tree
Definition	Each node has ≤ 2 children	Binary Tree + BST property
BST Property	None	Left < Root < Right
Search	$O(n)$	$O(\log n)$ avg, $O(n)$ worst
Insertion	$O(n)$	$O(\log n)$ avg, $O(n)$ worst
In-order	Random	Sorted (ascending) output
Use case	General hierarchy	Search, dictionaries

BST + Pre-order + Post-order: Given a BST, you can perform pre-order (Root-L-R) and post-order (L-R-Root) traversals. Combined with in-order you can uniquely reconstruct it.

Q. Explain Binary Tree – find the visited path / traversal of a given tree.

Given any tree, apply the traversal rules:

- Pre-order: Visit node, then recurse Left, then recurse Right.
- In-order: Recurse Left, visit node, recurse Right.
- Post-order: Recurse Left, recurse Right, visit node.

Example Tree: Root=A, Left=B (children D,E), Right=C (child F)

Pre-order: A B D E C F

In-order: D B E A C F

Post-order: D E B F C A

Q. What is length of the tree? What is the visited/weighted path?

Length of tree = number of edges on the **longest path from root to leaf** (i.e., the height of the tree).

Height = max depth of any leaf node

The **weighted path** (weighted path length) = sum of (weight × depth) for each node, used in Huffman trees.

Q. What is an AVL Tree? Explain Balance Factor, rotations, and insertion with example keys: 4, 3, 1, 2, 7, 6, 11.

An **AVL Tree** is a self-balancing BST where the **Balance Factor** of every node is -1, 0, or +1.

Balance Factor (BF)

BF = Height(Left Subtree) – Height(Right Subtree)

- BF = 0 → Balanced
- BF = +1 → Left heavy (allowed)
- BF = -1 → Right heavy (allowed)
- |BF| > 1 → UNBALANCED → Rotation needed

Types of Rotations

Case	Rotation Type	When
LL Case	Right Rotation	New node in Left of Left
RR Case	Left Rotation	New node in Right of Right
LR Case	Left then Right	New node in Right of Left
RL Case	Right then Left	New node in Left of Right

Step-by-step insertion of 4, 3, 1, 2, 7, 6, 11:

- Insert 4 → Root. BF=0. Balanced.
- Insert 3 → Left of 4. BF(4)=+1. Balanced.
- Insert 1 → Left of 3. BF(3)=+1, BF(4)=+2. UNBALANCED! LL case → Right Rotate at 4.

After rotation: Root=3, Left=1, Right=4

- Insert 2 → Right of 1. BF(1)=-1, BF(3)=0. Balanced.
- Insert 7 → Right of 4. BF(4)=-1, BF(3)=-1. Balanced.
- Insert 6 → Left of 7. BF(7)=+1, BF(4)=-2. UNBALANCED! RL case → Left Rotate 7, Right Rotate 4.
- Insert 11 → Right of 7. Rebalance as needed.

■ Reference: cs.usfca.edu has an interactive AVL tree visualizer.

■ ■ **BF = LH – RH. If BF = 2 → Left heavy unbalanced. If BF = -2 → Right heavy unbalanced.**

2-3 Tree and 2-3-4 Tree (Balanced BSTs)

- **2-3 Tree:** Each node has 2 or 3 children, all leaves at same level.
- **2-3-4 Tree:** Each node has 2, 3, or 4 children. Equivalent to Red-Black tree.
- **Red-Black Tree:** Self-balancing BST with color properties ensuring $O(\log n)$ operations.

GRAPHS: BFS & DFS

Traversal · Algorithms · Order · Data Structures Used

Q. Explain BFS (Breadth First Search) and DFS (Depth First Search). From a given node, find traversal order and path.

Graph: A, B, C, D, E, F, G, H

Edges (from diagram): A-B, A-C, B-D, C-E, D-G, D-F, E-F, F-H

DFS – Depth First Search

Explores as far as possible along each branch before backtracking. Uses a **STACK** (explicit or call stack via recursion).

Data Structure Used: STACK

DFS Algorithm:

```
DFS(graph, start):  
    push start onto stack; mark visited  
    while stack not empty:  
        node = stack.pop()  
        for each neighbor of node:  
            if not visited: push neighbor; mark visited
```

DFS from node A (example graph):

A → B → D → G → F → E → C → H

- Order depends on adjacency list ordering.

BFS – Breadth First Search

Explores all neighbors at current depth before going deeper. Uses a **QUEUE** as its data structure.

Data Structure Used: QUEUE

BFS Algorithm:

```
BFS(graph, start):  
    enqueue start; mark visited  
    while queue not empty:  
        node = queue.dequeue()  
        for each neighbor of node:  
            if not visited: enqueue neighbor; mark visited
```

BFS from node A (example graph):

A → B → C → D → E → G → F → H

BFS vs DFS Comparison

Feature	BFS	DFS
Full Form	Breadth First Search	Depth First Search
Data Structure	Queue	Stack (or Recursion)
Traversal	Level by level	Branch by branch
Space	$O(V)$ – queue size	$O(V)$ – stack depth
Time	$O(V + E)$	$O(V + E)$
Shortest Path	Yes (unweighted)	Not guaranteed
Completeness	Complete	Complete (finite graphs)

■■ In exam: A random starting node will be given. Apply BFS and DFS and write the traversal order step by step.

Q. Graph traversal: explain the order and which data structure is used for BFS/DFS on a given graph. Also relate to in-order, pre-order.

For **Trees** (special case of graph):

- DFS on tree = Pre-order / In-order / Post-order traversal.
 - BFS on tree = Level-order traversal (level by level).
-

DYNAMIC PROGRAMMING, GREEDY & DIVIDE-AND-CONQUER

Which gives optimal solution?

Q. Dynamic Programming vs Greedy Algorithm – which one gives the optimal solution? Explain with examples and approach.

Dynamic Programming (DP)

Definition: Solves complex problems by breaking them into **overlapping subproblems** and storing results (memoization/tabulation) to avoid recomputation.

- Approach: Top-down (memoization) or Bottom-up (tabulation).
- Gives GLOBALLY OPTIMAL solution.
- Examples: Fibonacci, Knapsack 0/1, Longest Common Subsequence, Floyd-Warshall.

Time: Usually $O(n^2)$ or $O(n \times W)$ | Space: $O(n)$ to $O(n^2)$

Greedy Algorithm

Definition: Makes the **locally optimal choice** at each step hoping to find global optimum. Does NOT revisit decisions.

- Approach: Always pick the best available option right now.
- May NOT give optimal solution for all problems.
- Examples: Activity Selection, Huffman Coding, Dijkstra's, Prim's, Kruskal's, Coin Change (standard denominations).

Time: Usually $O(n \log n)$ | Space: $O(1)$ to $O(n)$

DP vs Greedy – Head to Head

Feature	Dynamic Programming	Greedy Algorithm
Optimal?	Always optimal	Not always optimal
Subproblems	Overlapping	Non-overlapping
Decision	All possibilities considered	One best local choice
Backtracking	Yes (implicit)	No
Space	$O(n)$ to $O(n^2)$ table	$O(1)$ usually
Speed	Slower	Faster
Example	0/1 Knapsack	Fractional Knapsack

■ DP always gives GLOBALLY OPTIMAL solution. Greedy gives local optimum (may or may not be global).

Divide and Conquer

Definition: Divides problem into **non-overlapping** subproblems, solves each independently, and combines results.

- Examples: Merge Sort, Quick Sort, Binary Search, Strassen's Matrix.
- Difference from DP: subproblems do NOT overlap.

Paradigm	Subproblems	Example	Optimal?
Divide & Conquer	Non-overlapping	Merge Sort, Quick Sort	Yes for sorting
Dynamic Programming	Overlapping	Knapsack, Fibonacci	Always
Greedy	Sequential choices	Huffman, Dijkstra	Problem-specific

CHAPTER 8

QUICK REFERENCE & EXAM TIPS

All Complexities · Must-Know Facts · Common Traps

ALL SORTING COMPLEXITIES – ASCENDING ORDER

Rank	Algorithm	Worst Case	Note
1 (Fastest)	Counting/Radix Sort	$O(n+k)$	Non-comparison based
2	Merge Sort	$O(n \log n)$	Stable, extra space
3	Heap Sort	$O(n \log n)$	In-place, not stable
4	Quick Sort	$O(n \log n)$ avg / $O(n^2)$ worst	Usually fastest in practice
5	Insertion Sort	$O(n^2)$	Best for nearly sorted
6	Bubble Sort	$O(n^2)$	Simple, slow
7 (Slowest)	Selection Sort	$O(n^2)$	Always $O(n^2)$

STACK vs QUEUE – QUICK FACTS

Property	Stack	Queue
Principle	LIFO (Last In First Out)	FIFO (First In First Out)
Insert	Push (at top)	Enqueue (at rear)
Delete	Pop (from top)	Dequeue (from front)
Peek	Top element	Front element
Overflow	$top == MAX-1$	$(rear+1)\%MAX == front$
Underflow	$top == -1$	$front == -1$
Used in	DFS, recursion	BFS, scheduling

TREE TRAVERSAL – MEMORY AID

Traversal	Order	Memory Tip
Pre-order	Root $\rightarrow$ L $\rightarrow$ R	'PRE' = Root is BEFORE children
In-order	L $\rightarrow$ Root $\rightarrow$ R	'IN' = Root is IN BETWEEN children
Post-order	L $\rightarrow$ R $\rightarrow$ Root	'POST' = Root is AFTER children

IMPORTANT EXAM TIPS

- 1. For array-based merge/quick sort question: show EACH STEP of partition or merge.
- 2. For AVL: always calculate BF after each insertion and show rotation type.
- 3. For BFS/DFS: always mention the data structure used (Queue/Stack).
- 4. For DP vs Greedy: DP = optimal always. Greedy = may not be optimal.

■ ■ 5. For complexity loops: count iterations mathematically, don't guess.

COMPLEXITY CHEAT SHEET

$$O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

Big-O	Name	Example
$O(1)$	Constant	Array access, Push/Pop
$O(\log n)$	Logarithmic	Binary Search, BST ops
$O(\sqrt{n})$	Square Root	Jump Search, Sieve
$O(n)$	Linear	Linear Search, single loop
$O(n \log n)$	Linearithmic	Merge Sort, Heap Sort
$O(n^2)$	Quadratic	Bubble/Selection/Insertion Sort
$O(2^n)$	Exponential	Naive Fibonacci, subsets
$O(n!)$	Factorial	Permutations, brute-force TSP

SET-2 SPECIAL TOPICS

1. Circular Queue using array:

Implement with front, rear pointers. Use modulo arithmetic. Show enqueue/dequeue with wrap-around example.

2. Linked List – add/delete at first, last, specific position:

Always draw the pointer changes. Show before/after node links.

3. Compare all sorts (Insertion, Merge, Quick, Bubble, Selection):

Use the master table above. For the program question, write code + derive T.C. line by line.

4. Quick Sort pivot selection:

If asked 'kr value ho to kis way se sort kare' → choose the right pivot strategy. Median-of-three pivot avoids worst case.

5. Balance Binary Tree types:

- 2-3 tree → Nodes have 2 or 3 children.
- 2-3-4 tree → Nodes have 2, 3, or 4 children.
- Red-Black tree → Binary tree with color constraints.
- AVL tree → Balance factor must be -1, 0, or +1.

6. Bubble/Insertion/Selection sort – one element insert:

Given a nearly sorted array (data is sorted), explain how ONE new element is inserted step by step:

- Insertion Sort: start from right, shift all larger elements right, insert.
- Time for one insert into sorted array: $O(n)$ worst case.

7. Dynamic Programming, Greedy, Divide & Conquer – Example:

DP: 0/1 Knapsack – list items, weights, values, build table.

Greedy: Activity Selection – sort by finish time, greedily pick.

D&C: Merge Sort – show divide and conquer steps on an example array.

